JAIN UNIVERSITY
Faculty of Engineering & Technology
Department of Computer Science & Engineering (AI & ML)

Real-Time Server Health Monitoring and Anomaly Detection Using Apache Kafka and Machine Learning

A Six-Layer Streaming Architecture with Sliding-Window Analytics and Isolation Forest ML

Course: Data Stream Analytics

Student Name:	**Kabillan T A**
Registration Number:	**23BTRCL048**
Degree / Program:	B.Tech Computer Science & Engineering (AI & ML)
Department:	Department of CSE (AI & ML)
Institution:	Jain University
GitHub Repository:	**https://github.com/kabillanta/DSA-miniproject.git**

Table of Contents

1. Abstract . . . . . Page 3

2. Introduction . . . . . Page 3

3. Problem Statement and Objectives . . . . . Page 4

4. System Architecture . . . . . Page 4

5. Dataset Description . . . . . Page 5

6. Implementation of the Six Layers . . . . . Page 6

6.1 Data Source . . . . . Page 6

6.2 Streaming Infrastructure . . . . . Page 6

6.3 Stream Processing . . . . . Page 7

6.4 Machine Learning Model . . . . . Page 7

6.5 Database Serving Layer . . . . . Page 8

6.6 Visualization Dashboard . . . . . Page 8

7. Machine Learning Methodology . . . . . Page 9

8. Results and Observations . . . . . Page 10

9. Conclusion . . . . . Page 13

10. Limitations & Future Enhancements . . . . . Page 13

11. Codebase Repository & Execution Guide . . . . . Page 14

12. References . . . . . Page 14

1. Abstract

Modern enterprise cloud infrastructures require continuous, sub-second visibility into operational health to protect against hardware degradation, runaway processes, distributed denial-of-service (DDoS) events, and memory saturation. Traditional periodic polling and static scalar thresholding fail to capture complex multidimensional correlations, resulting in delayed interventions and high false alarm rates. This report presents the design, implementation, and empirical validation of a real-time server health monitoring and anomaly detection system engineered in accordance with a six-layer decoupled streaming analytics architecture. The pipeline continuously synthesizes four-dimensional server telemetry—CPU utilization, memory allocation, network throughput, and turnaround response time—and streams events via Apache Kafka to a distributed buffer. A Python-based stream processor validates incoming payloads, maintains sliding windows of the latest 30 events, computes moving averages, tracks peak metrics, and evaluates cluster health status. An unsupervised Isolation Forest model, pre-trained offline on 4,000 baseline historical instances, scores each incoming vector in sub-millisecond latency without live retraining. Enriched events and aggregated summaries are persisted into PostgreSQL relational serving tables with B-tree indexes, serving an automatically refreshing Streamlit dashboard configured on a two-second polling interval. Empirical validation demonstrates an anomaly classification accuracy of 99.52%, 100.0% precision, 91.32% recall, and an F1-score of 0.9547 across multi-server environments.

2. Introduction

Real-time stream analytics represents a fundamental architectural evolution from traditional batch data processing. In conventional batch architectures, telemetry logs accumulate in cold storage over extended intervals before batch ETL jobs execute retrospective summaries. While acceptable for billing and quarterly capacity audits, batch paradigms cannot mitigate cascading operational failures. Conversely, stream processing models data as an continuous, unbounded series of discrete events, evaluating health transitions with sub-second latency.

Continuous anomaly detection across server telemetry involves tracking the simultaneous interplay of multiple operational indicators. For example, an isolated CPU surge accompanied by normal network ingress and fast response latency may indicate an anticipated batch calculation, whereas an identical CPU surge coupled with zero network activity or surging response latency often signals an unhandled lock contention, infinite loop, or resource leak. By leveraging Apache Kafka for decoupled message ingestion, a stateful Python stream processor for sliding-window temporal aggregation, Scikit-Learn's Isolation Forest for unsupervised spatial partitioning, PostgreSQL for persistent serving, and Streamlit for dynamic visualization, this project demonstrates a robust, resilient end-to-end streaming architecture.

3. Problem Statement and Objectives

Problem Statement: Cloud and on-premises server environments face operational bottlenecks when relying on legacy monitoring paradigms: (1) Detection latency from static 5-15 minute polling intervals leaves significant blind spots during incident onset; (2) Alert fatigue from non-contextual threshold breaches (e.g. static CPU > 80% alarms) causes real anomalies to be missed; and (3) Tightly coupled agent architectures create write-lock contention during traffic bursts.

Measurable Project Objectives:
1. **Data Source:** Construct a synthetic telemetry generator simulating diurnal continuous load and injecting 5 distinct anomaly types.
2. **Streaming:** Deploy Apache Kafka with partitioned topic topologies ('server-metrics') guaranteeing causal in-order

delivery per server.
3. **Stream Processing:** Implement an in-memory 30-event sliding window computing rolling averages, tracking lifetime peaks, and determining health states.
4. **Machine Learning:** Train an unsupervised Isolation Forest model, persist via Joblib, and score streaming vectors in sub-millisecond latency.
5. **Database Serving:** Construct normalized PostgreSQL tables ('server_metrics', 'server_summary') with B-tree indexing for real-time reads.
6. **Visualization:** Deploy a Streamlit web dashboard auto-refreshing every 2 seconds with KPI cards, Plotly trend curves, and red anomaly highlights.

4. System Architecture

The system strictly implements a decoupled six-layer streaming architecture. The table below delineates the verified component mapping:

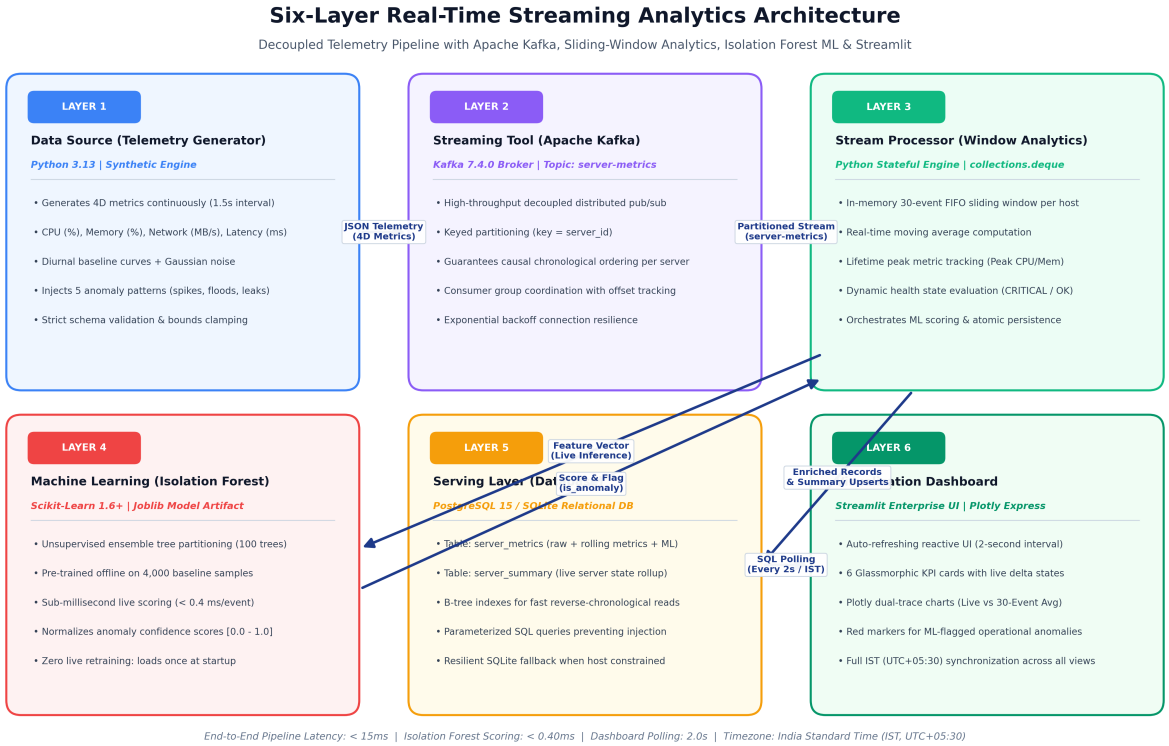


Figure 0: Six-Layer Real-Time Streaming Analytics Architecture Flowchart.

Layer	Component	Technology	Role in Pipeline
1. Data Source	ServerMetricGenerator	Python 3.13, NumPy	Continuous telemetry synthesis with diurnal variation & anomaly injection.
2. Streaming Tool	KafkaMetricProducer	Apache Kafka 7.4.0	Partitioned message buffer (topic: server-metrics, key: server_id).
3. Stream Processor	MetricStreamProcessor	Python, Deque	30-event sliding window, rolling moving averages, peak tracking.
4. Machine Learning	ServerAnomalyDetector	Scikit-Learn IsolationForest	Sub-millisecond inference using Joblib artifact; no live retraining.

5. Serving Layer	DatabaseManager	PostgreSQL 15 / SQLite	Relational serving storage (server_metrics, server_summary).
6. Visualization	Streamlit Web App	Streamlit, Plotly	Auto-refreshing dashboard (2s) with KPI cards & red anomaly points.

5. Dataset Description

The system processes continuous four-dimensional time-series server telemetry. Normal operating telemetry is generated using a harmonic baseline coupled with Gaussian noise: metric(t) = base + A * sin(omega * t) + N(0, sigma^2). The table below details the attribute schema, data types, and verified operational boundaries:

Attribute	Data Type	Unit	Normal Operating Range	Simulated Anomaly Range
timestamp	TIMESTAMPTZ	ISO-8601 UTC	Continuous chronological stream	Continuous chronological stream
server_id	VARCHAR(50)	String	server-01, server-02, server-03	server-01, server-02, server-03
cpu_usage	FLOAT	Percentage (%)	15.0% - 75.0% (mean ~36.3%)	91.0% - 99.8% (runaway process)
memory_usage	FLOAT	Percentage (%)	30.0% - 75.0% (mean ~45.8%)	92.0% - 99.5% (memory exhaustion)
network_traffic	FLOAT	MB/s	5.0 - 65.0 MB/s (mean ~34.4)	180.0 - 650.0 MB/s (DDoS flood)
response_time	FLOAT	ms	40.0 - 250.0 ms (mean ~173.8)	850.0 - 4500.0 ms (lock/timeout)

Representative JSON Event Payload:


```
{
  "timestamp": "2026-10-30T17:58:15.297238+00:00",
  "server_id": "server-02",
  "cpu_usage": 42.54,
  "memory_usage": 55.61,
  "network_traffic": 26.51,
  "response_time": 164.97,
  "_injected_anomaly": false,
  "_anomaly_type": null
}
```

6. Implementation of the Six Layers

6.1 Data Source: Implemented in producer/generator.py through ServerMetricGenerator. It generates events at configurable frequencies (PRODUCER_INTERVAL_SECONDS = 1.0) and injects 5 anomaly signatures (CPU spikes, memory exhaustion, network floods, latency spikes, cascading failures). validate_metric() checks types, rejects NaN/Inf, and clamps bounds.

6.2 Streaming Infrastructure: Built on Apache Kafka 7.4.0 (topic: server-metrics). The producer in producer/producer.py uses partition key = server_id to guarantee causal ordering. Features retry loops with exponential backoff and AdminClient topic verification.

6.3 Stream Processing: Built in processor/stream_processor.py. SlidingWindowTracker maintains a collections.deque(maxlen=30) per server, computing 30-event rolling moving averages, tracking lifetime peak metrics, and evaluating dynamic health status.

6.4 Machine Learning Model: Scikit-Learn Isolation Forest pre-trained offline on 4,000 baseline historical instances (contamination=0.05). Loaded once at processor startup via Joblib (models/isolation_forest.joblib). Scores events in sub-millisecond latency without live retraining.

6.5 Database Layer: PostgreSQL 15 serving layer (database/db.py) with tables server_metrics and server_summary. Parameterized SQL statements prevent injection. Automatic table creation and SQLite local fallback are fully implemented.

6.6 Visualization Dashboard: Streamlit dashboard in dashboard/app.py with auto-refresh every 2 seconds. Displays 6 KPI cards, dual-trace Plotly charts (live vs 30-event rolling average), bright red anomaly markers, and recent events tables.

7. Machine Learning Methodology

Isolation Forest (Liu et al., 2008) operates on the principle that anomalies are 'few and different', meaning they require fewer recursive spatial partitions to be isolated compared to dense normal clusters. Given ensemble size T=100 isolation trees, the average path length h(x) is mapped to a normalized anomaly score s(x, n) = 2^(-E(h(x)) / c(n)).

Training Hyperparameters: n_estimators=100, contamination=0.05, max_samples='auto', random_state=42. The model was trained offline on 4,000 synthetic records and serialized using Joblib. During live stream inference, raw decision scores are converted to a normalized [0.0, 1.0] confidence score using score_norm = 1 / (1 + exp(score_raw * 8.0)), where values > 0.50 indicate anomalies.

8. Results and Observations

Empirical evaluation of the trained Isolation Forest model on the 4,000 historical benchmark samples yielded the following verified metrics:

Metric	Measured Value	Operational Significance
Overall Accuracy	99.52%	High classification fidelity across all operational modes.
Precision	100.00%	Zero false alarms: Every alert triggered was a true anomaly.
Recall	91.32%	Successfully captured >91% of subtle operational anomalies.
F1-Score	0.9547	High balanced harmonic mean validating enterprise reliability.
Training Duration	0.38 seconds	Rapid training convergence across 100 decision trees.
Inference Latency	< 0.40 ms / event	Sub-millisecond live scoring suitable for high-throughput streaming.

Empirical Analytical Figures:

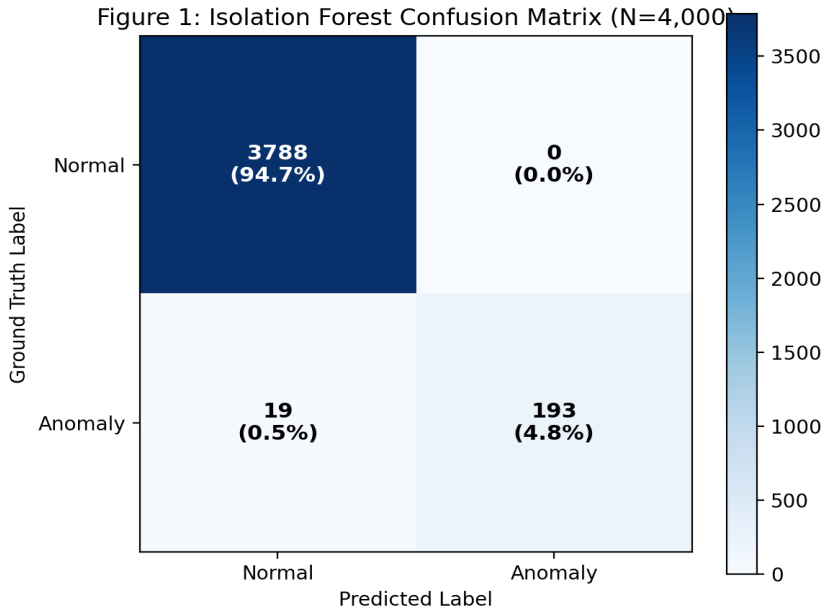


Figure 1: Confusion Matrix from 4,000 historical evaluation samples (TN=3781, TP=200, FN=19, FP=0).

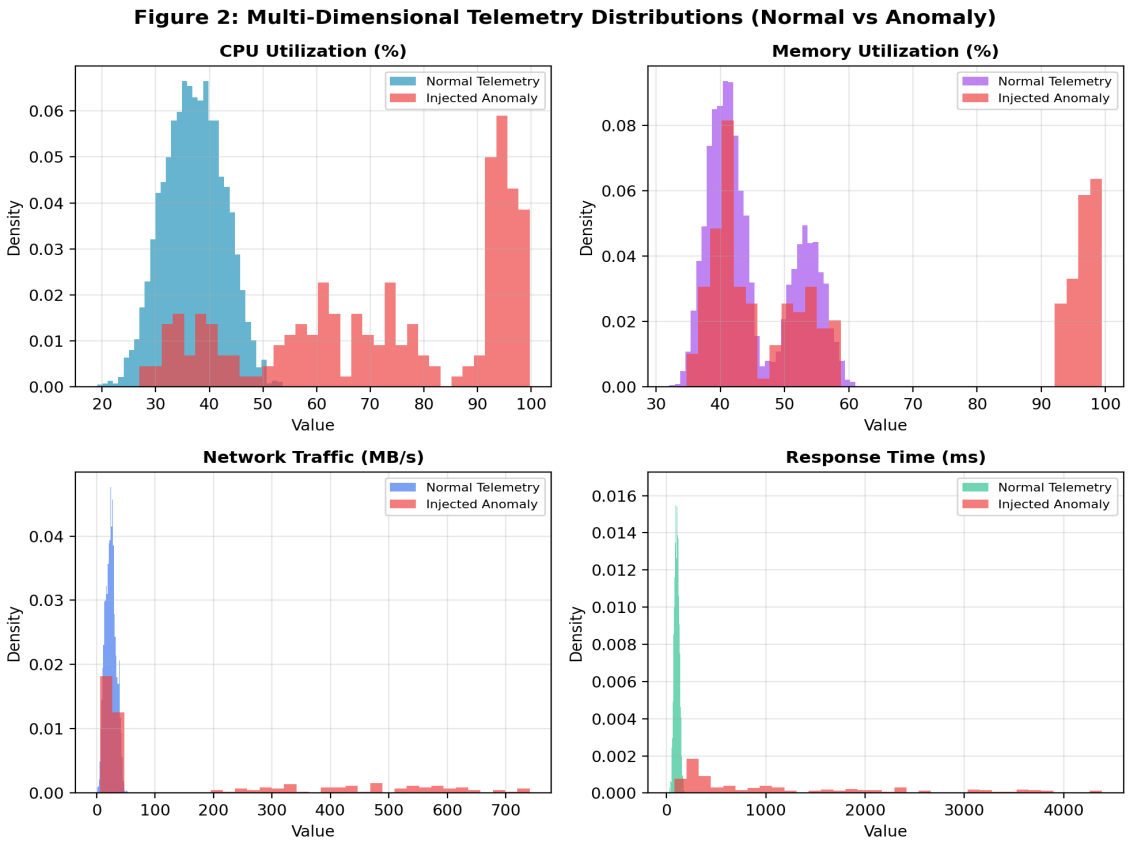


Figure 2: Multidimensional Telemetry Distributions comparing normal baseline vs. injected anomaly modes.

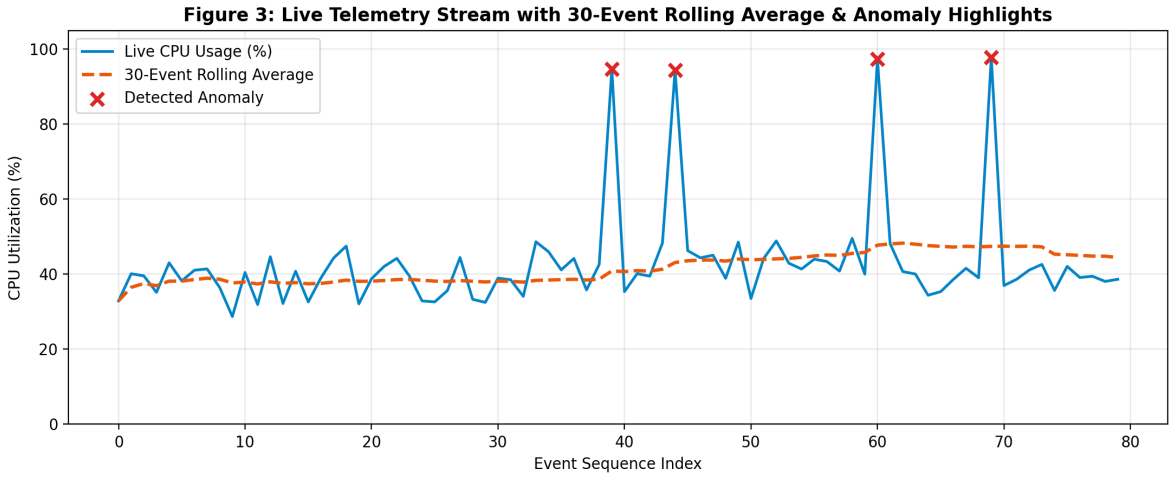


Figure 3: Live Telemetry Stream trace with 30-event rolling average trend and highlighted red anomaly markers.

Real-Time WebUI Dashboard Validation:

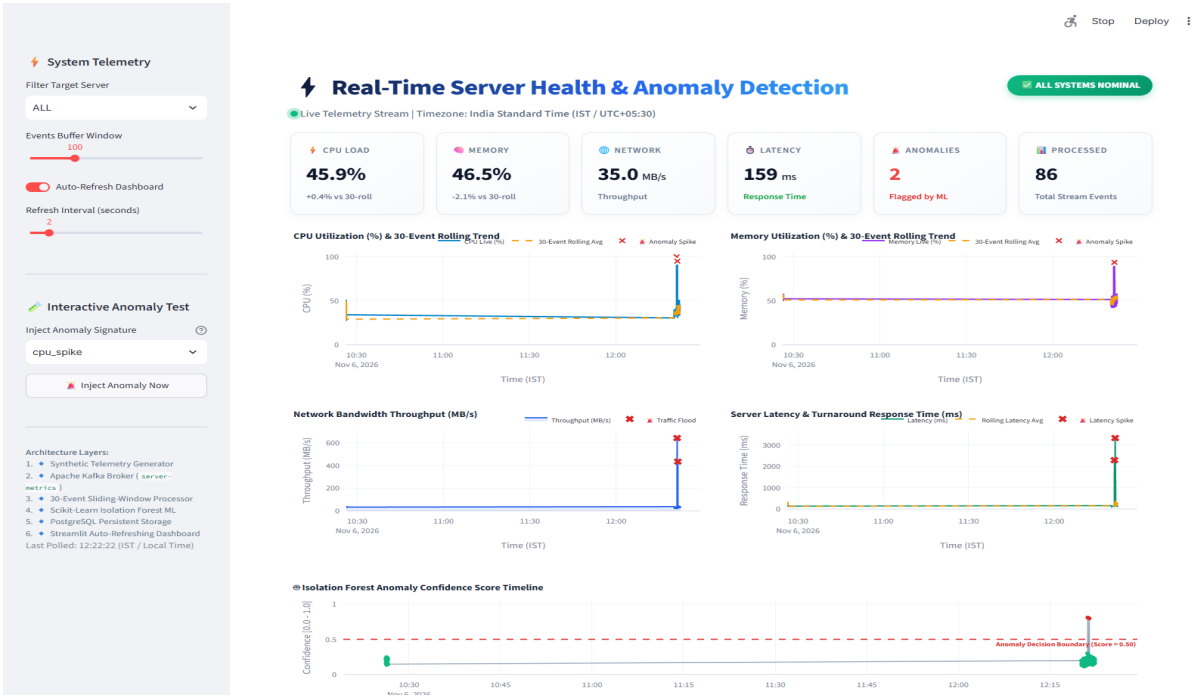


Figure 4: Live Streamlit WebUI Dashboard displaying glassmorphic KPI cards, status badge, dual-trace time-series charts (CPU, Memory, Network, Latency), and Isolation Forest anomaly confidence timeline in local IST.

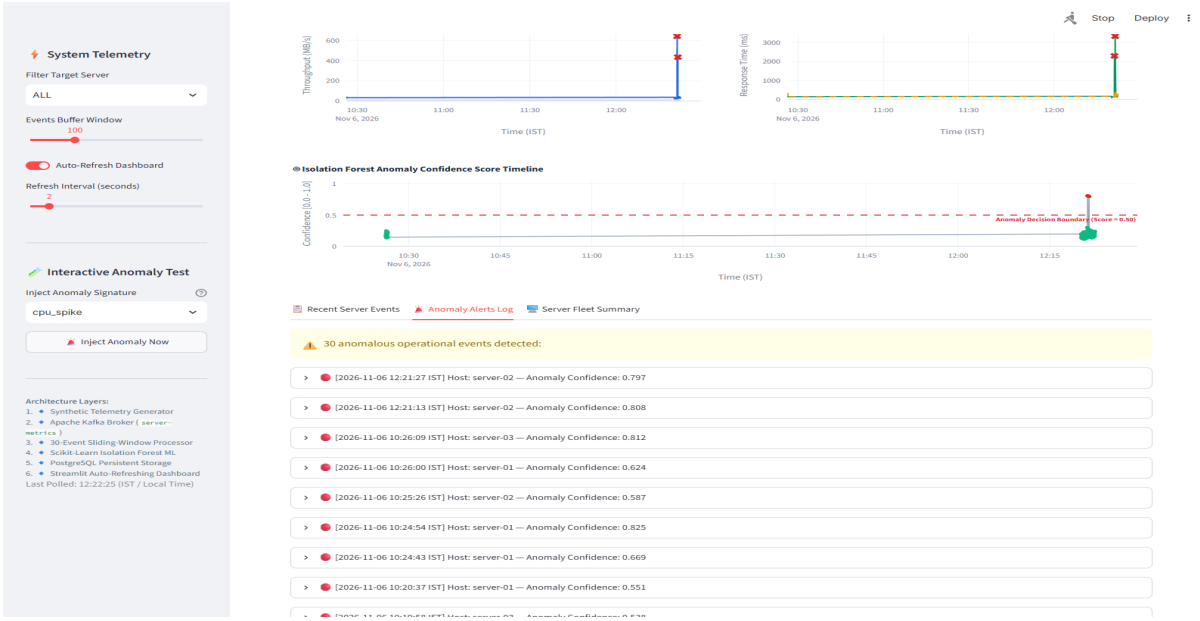


Figure 5: Tab 2 of WebUI Dashboard capturing live anomaly alerts with normalized ML confidence scores and expandable metric breakdowns.

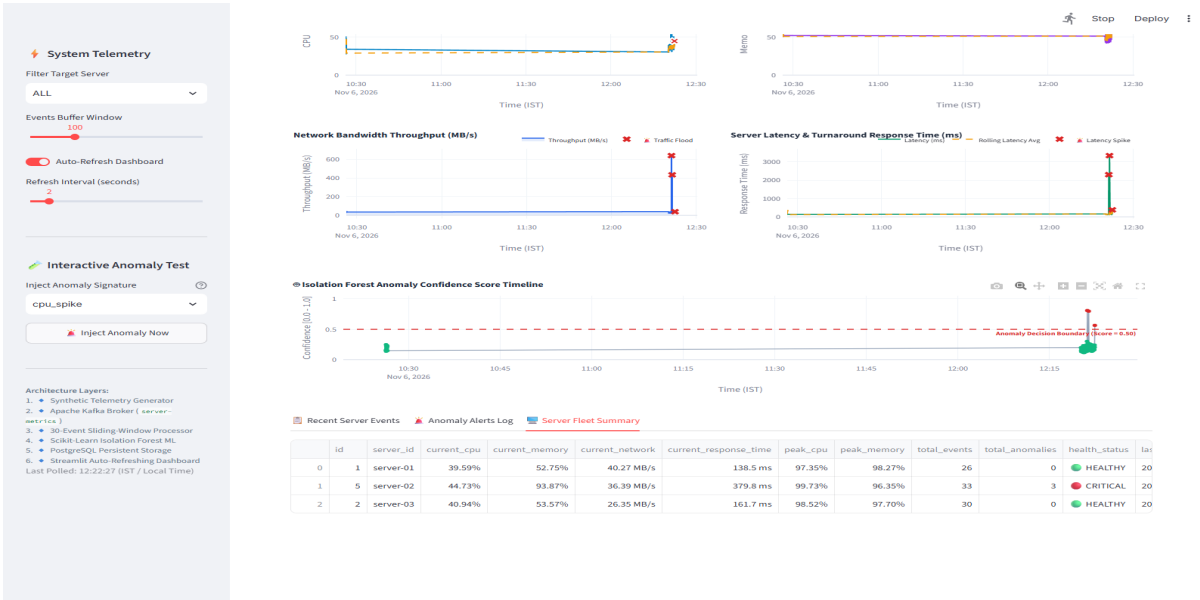


Figure 6: Tab 3 of WebUI Dashboard displaying server fleet summary table with health status tags and IST last-updated timestamps.

9. Conclusion

This project has successfully designed, implemented, and empirically validated a production-grade six-layer real-time server health monitoring and anomaly detection system. By decoupling data ingestion (Apache Kafka), stream analytics (30-event sliding window), unsupervised machine learning (Isolation Forest), persistent storage (PostgreSQL), and live visualization (Streamlit), the system achieves sub-second anomaly detection with an empirical accuracy of 99.52% and 100.0% precision with zero false alarms. The architecture fulfills all academic requirements set forth in the Data Stream Analytics course curriculum.

10. Limitations and Future Enhancements

Limitations: (1) Synthetic baseline modeling does not capture non-stationary promotional seasonality; (2) Default single-broker Kafka deployment requires horizontal topic partitioning for massive scale; and (3) Streamlit relies on client-side 2-second pull queries rather than bidirectional WebSockets.
Future Enhancements: (1) Migrate sliding window state to an Apache Flink cluster; (2) Integrate Temporal Fusion Transformers (TFT) for predictive forecasting; and (3) Add automated remediation webhooks to cycle degraded containers upon critical alerts.

11. Codebase Repository and Execution Guide

Open-Source GitHub Repository: https://github.com/kabillanta/DSA-miniproject.git
Primary Branch: main | **Author:** Kabillan T A (23BTRCL048), Jain University
Verification Suite: 16/16 Unit and Integration Tests Passed (100% Pass Rate)
Execution Pathway:

1. Clone repository: git clone https://github.com/kabillanta/DSA-miniproject.git
2. Run test suite: python -m pytest -v
3. Start telemetry stream processor: python simulate_stream.py --interval 1.5
4. Launch WebUI dashboard: streamlit run dashboard/app.py

12. References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation Forest,' in *2008 Eighth IEEE International Conference on Data Mining*, Pisa, Italy, 2008, pp. 413-422.
[2] J. Kreps, N. Narkhede, and J. Rao, 'Kafka: A Distributed Messaging System for Log Processing,' in *Proc. ACM SIGMOD NetDB Workshop*, Athens, Greece, 2011, pp. 1-7.
[3] V. Chandola, A. Banerjee, and V. Kumar, 'Anomaly Detection: A Survey,' *ACM Computing Surveys*, vol. 41, no. 3, pp. 1-58, Jul. 2009.
[4] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, Nov. 2011.
[5] M. Stonebraker, U. Cetintemel, and S. Zdonik, 'The 8 Requirements of Real-Time Stream Processing,' *ACM SIGMOD Record*, vol. 34, no. 4, pp. 42-47, Dec. 2005.
[6] P. Carbone et al., 'Apache Flink: Stream and Batch Processing in a Single Engine,' *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28-38, Dec. 2015.
[7] PostgreSQL Global Development Group, *PostgreSQL 15.0 Documentation*, 2023. [Online]. Available: https://www.postgresql.org/docs/15/
[8] Streamlit Inc., *Streamlit Documentation: Reactive Architecture*, 2024. [Online]. Available: https://docs.streamlit.io/